Artificial Intelligence

AI-2002

Lab 05

Local Search (Beam Search, Hill Climbing, Genetic Algorithm)

National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2002	Artificial Intelligence Lab

1. Objective 3
2. Local Search 3
2.1 Types Of Local Search 3
2.1.1 Beam Search 4
2.1.2 Hill Climbing 6
2.1.3 Genetic Algorithm 9
3. Lab Tasks 19

1. Objective

- Learn how to design and implement local search, beam search, hill climbing, and genetic algorithms to solve optimization problems.
- Evaluate the efficiency, strengths, and limitations of each search technique through practical examples and compare their performance.
- Apply these search methods to find optimal or near-optimal solutions for problems like the N-Queens.
- Develop skills in formulating a fitness function and evaluating candidate solutions to drive population evolution towards optimal solutions.

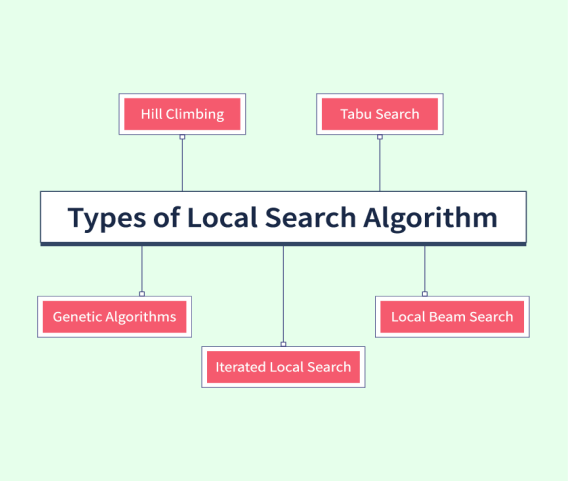
2. Local Search

Local Search refers to an optimization technique used for solving computational problems where the goal is to **find an optimal or near-optimal solution by exploring neighboring states iteratively**. Unlike global search algorithms, which explore the entire solution space, local search methods work by starting from an initial candidate solution and iteratively improving it by making small modifications. These algorithms are particularly useful for problems with large or infinite state spaces.

2.1 Types Of Local Search

1. **Hill Climbing:** An iterative algorithm that starts with an arbitrary solution and continuously makes small changes to improve it, moving "uphill" in terms of the objective function. It can get stuck in local maxima, plateaus, or ridges due to its greedy nature.

2. **Simulated Annealing:** Inspired by the annealing process in metallurgy, this algorithm allows occasional moves to worse solutions to escape local optima. It gradually reduces the likelihood of such moves as the search progresses, balancing exploration and exploitation.



3. **Genetic Algorithms:** A population-based search method inspired by natural selection, where potential solutions evolve through selection, crossover, and mutation. It explores multiple paths in parallel, reducing the chances of getting trapped in local optima.

4. **Beam Search:** Maintains a limited set of the most promising partial solutions at each step, expanding them based on a heuristic. Unlike greedy search, it explores multiple paths simultaneously but with a fixed-width beam to manage memory and computational cost.

2.1.1 Beam Search

Beam Search is a heuristic search algorithm that navigates a search space by systematically expanding the most promising nodes within a constrained set. This approach combines elements of breadth-first search to construct its search tree by generating all successors at each level. However, **it only evaluates and expands a set number, W, of the best nodes at each level, based on their heuristic values**. This selection process is repeated at each level of the tree. Beam Search explores **multiple paths in parallel** but only retains a limited set of top paths at each step.

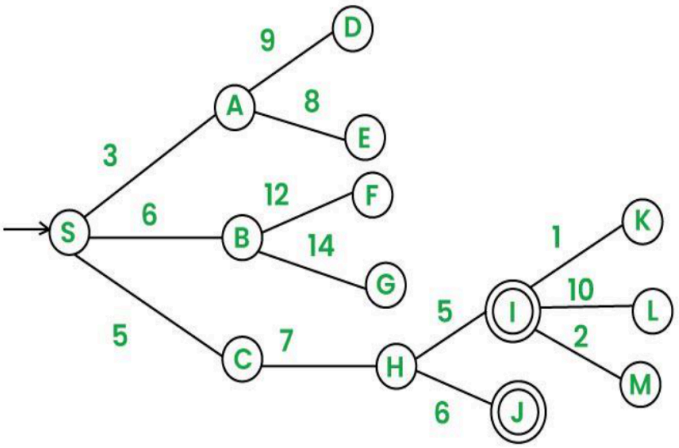
Key Concepts in Beam Search

1. **Beam Width *(k)*:** A predefined number that limits how many partial solutions (or paths) are retained at each level of the search. Larger beam widths explore more possibilities but require more computation.
2. **Heuristic Score:** Each path is evaluated based on a heuristic function that estimates its likelihood or quality.
3. **Pruning:** Paths that do not make it into the top *k* at each step are discarded.

Beam Search Algorithm

1. Initialize the beam with the start state.
2. Set the beam width *k* to a predefined value.
3. While the termination condition is not met:
    - Generate all possible successor states for each state in the beam.
    - Calculate a heuristic score for each successor state.
    - Rank the successor states based on their scores.
    - Retain the top *k* states and discard the rest.
    - Update the beam with the retained states.
4. Return the best path based on the final scores.

```
graph = {
    'S': [('A', 3), ('B', 6), ('C', 5)],
    'A': [('D', 9), ('E', 8)],
    'B': [('F', 12),
('G', 14)],
    'C': [('H', 7)],
    'H': [('I', 5),
('J', 6)],
    'I': [('K', 1),
('L', 10), ('M', 2)],
    'D': [],'E': [],
    'F': [],'G': [],
    'J': [],'K': [],
    'L': [],'M': []
}
```




```
# Beam Search function
def beam_search(start, goal, beam_width=2):
    # Initialize the beam with the start state
    beam = [(0, [start])]  # (cumulative cost, path)

    while beam:
        candidates = []
        # Expand each path in the beam
        for cost, path in beam:
            current_node = path[-1]
            if current_node == goal:
                return path, cost  # Return the path and cost if goal
            # Generate successors
            for neighbor, edge_cost in graph.get(current_node, []):
                new_cost = cost + edge_cost
                new_path = path + [neighbor]
                candidates.append((new_cost, new_path))

        # Select top-k paths based on the lowest cumulative cost
        beam = heapq.nsmallest(beam_width, candidates, key=lambda x:
x[0])
        #print(beam)
```

```
    return None, float('inf')  # Return None if no path is found

# Run Beam Search
start_node = 'S'
goal_node = 'L'
beam_width = 3
path, cost = beam_search(start=start_node, goal=goal_node,
beam_width=beam_width)

# Print results
if path:
    print(f"Path found: {' → '.join(path)} with total cost: {cost}")
else:
    print("No path found.")
```

2.1.2 Hill Climbing Search

Hill Climbing is a heuristic search algorithm used primarily for mathematical optimization problems in artificial intelligence (AI). It is a form of local search, which means it **focuses on finding the optimal solution by making incremental changes to an existing solution** and then evaluating whether the new solution is better than the current one. The process is analogous to climbing a hill where you continually seek to improve your position until you reach the top, or local maximum, from where no further improvement can be made.

Types of Hill Climbing

1. **Simple Hill Climbing:** Evaluates each neighbor one by one and moves to the first better neighbor.
2. **Steepest-Ascent Hill Climbing:** Evaluates all neighbors and moves to the best one.
3. **Stochastic Hill Climbing:** Randomly selects a neighbor and moves to it if it's better.

Simple Hill Climbing Algorithm

1. Start at the initial node and evaluate its heuristic score.
2. Generate one neighboring node at a time.
3. If the neighbor's score is better than the current node, move to that neighbor, else generate the next neighbor.

4. If no neighbor has a better score, terminate the search.
5. Return the current node as the solution.

Limitations of Hill Climbing

1. **Local Maxima:** Can get stuck at a peak that isn't the highest.
2. **Plateaus:** May halt if all neighbors have the same score.
3. **Ridges:** Can struggle with narrow paths to better solutions.

Solving N-Queens Using Simple Hill Climbing

We aim to place N queens on an N×N chessboard such that no two queens attack each other.

1. **State Representation:** Each row has one queen, and the column position is stored in a list.
   Example for *N=4*: [1, 3, 0, 2] means:
   - Queen in Row 0 is in Column 1.
   - Queen in Row 1 is in Column 3, and so on.
2. **Heuristic Function:** Counts the number of pairs of queens attacking each other.
3. **Neighbor Generation:** Moves a queen in a row to different columns to create neighbors.

```
import random
# Heuristic function: Counts the number of pairs of attacking queens
def calculate_conflicts(state):
    print(state)
    conflicts = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            # Check same column or diagonal
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i
- j):
                conflicts += 1
    return conflicts

# Generate neighbors by moving one queen at a time
def get_neighbors(state):
    neighbors = []
    n = len(state)
```

```
    for row in range(n):
        for col in range(n):
            if col != state[row]:
                new_state = list(state)
                new_state[row] = col
                neighbors.append(new_state)
    return neighbors

# Simple Hill Climbing function
def simple_hill_climbing(n):
    # Random initial state
    current_state = [random.randint(0, n - 1) for _ in range(n)]
    current_conflicts = calculate_conflicts(current_state)

    while True:
        neighbors = get_neighbors(current_state)
        next_state = None
        next_conflicts = current_conflicts
        # Find the first better neighbor
        for neighbor in neighbors:
            neighbor_conflicts = calculate_conflicts(neighbor)
            if neighbor_conflicts < next_conflicts:
                next_state = neighbor
                next_conflicts = neighbor_conflicts
                break  # Move to the first better neighbor

        # If no better neighbor is found, return the current state
        if next_conflicts >= current_conflicts:
            break

        # Move to the better neighbor
        current_state = next_state
        current_conflicts = next_conflicts

    return current_state, current_conflicts

# Run Simple Hill Climbing for N-Queens
n = 8  # Change N here for different sizes
solution, conflicts = simple_hill_climbing(4)
```

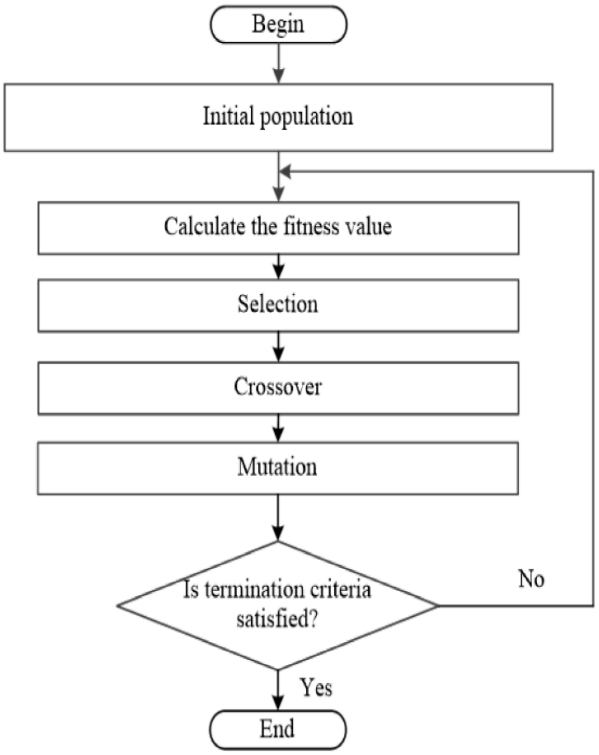
```
# Print results
if conflicts == 0:
    print(f"Solution found for {n}-Queens problem:")
    print(solution)
else:
    print(f"Could not find a solution. Stuck at state with {conflicts}
conflicts:")
    print(solution)
```

2.1.3 Genetic Algorithm

Genetic Algorithms(GAs) are adaptive heuristic search algorithms that belong to the larger part of evolutionary algorithms. Genetic algorithms simulate the process of natural selection which means those species that can adapt to changes in their environment can survive and reproduce and go to the next generation. In simple words, they simulate "survival of the fittest" among individuals of consecutive generations to solve a problem.

Key Concepts in Genetic Algorithms

1. **Population**: A set of candidate solutions (individuals) to the problem.
2. **Fitness Function**: A function that evaluates how good a candidate solution is. The goal is to maximize or minimize this function.
3. **Selection**: The process of choosing the fittest individuals from the population to produce offspring.
4. **Crossover (Recombination)**: Combining parts of two or more parent solutions to create new offspring.
5. **Mutation**: Randomly altering parts of a candidate solution to introduce diversity into the population.
6. **Generations**: Iterative steps where the population evolves over time to produce better solutions.



Convergence/Termination Criteria

Convergence criteria refer to the conditions or rules that determine when the algorithm should stop running. These criteria are used to decide whether the algorithm has found a satisfactory solution or if further iterations (generations) are unlikely to significantly improve the results.

Common Convergence Criteria in Genetic Algorithms:

1. **Maximum Number of Generations**:
   - The algorithm stops after a predefined number of generations.
   - Example: Stop after 1000 generations, regardless of the solution quality.
2. **Fitness Threshold**:
   - The algorithm stops when a solution achieves a fitness value above (or below, depending on the problem) a specified threshold.
   - Example: Stop when the best solution has a fitness score of 95% or higher.
3. **Stagnation (No Improvement):**
   - The algorithm stops if there is no significant improvement in the best fitness value over a certain number of generations.
   - Example: Stop if the best fitness value hasn't improved in the last 50 generations.
4. **Population Diversity:**
   - The algorithm stops if the population becomes too similar (low diversity), indicating that further exploration is unlikely to yield better solutions.
   - Example: Stop if 90% of the population has the same fitness value.
5. **Target Solution Found:**
   - The algorithm stops if a specific target solution is found.
   - Example: Stop if a solution matches a predefined configuration or value.

Solving N-Queens Using Genetic Algorithm

Step 1: Define the Problem and Fitness Function

Create a fitness function that counts the number of non-attacking pairs of queens.

- A higher fitness score means fewer conflicts (non-attacking pairs).
- Each queen must be placed in a different column, so the chromosome is a list of column positions for each row.

```
import random
# Fitness function: counts non-attacking pairs of queens
def calculate_fitness(individual):
    non_attacking_pairs = 0
    total_pairs = n * (n - 1) // 2  # Maximum possible non-attacking pairs

    # Check for conflicts
    for i in range(n):
        for j in range(i + 1, n):
            # No same column or diagonal conflict
            if individual[i] != individual[j] and abs(individual[i] -
individual[j]) != abs(i - j):
                non_attacking_pairs += 1

    # Fitness score is the ratio of non-attacking pairs
    return non_attacking_pairs / total_pairs
```

Step 2: Initialize the Population

This function generates a random route (a chromosome) by:

- Creating a list of indices representing the unique column position of each queen in a row (e.g., [3, 0, 2, 1]).

```
# Generate a random individual (chromosome) based on column positions
def create_random_individual():
    return random.sample(range(n), n)  # Ensure unique column positions
```

- This creates an **initial population** of **10 random boards**.
- Each board state is a potential solution, and the population is a collection of these solutions

```
# Create an initial population of random individuals
population_size = 10
population = [create_random_individual() for _ in range(population_size)]
```

Step 3: Evaluate Fitness

- Evaluates the fitness of each board state based on non-attacking pairs.
- In this problem, a higher fitness level is better.

- Fitness_scores is a list containing the fitness score for each board state in the population.

```
# Evaluate fitness for each route in the population
fitness_scores = [calculate_fitness(individual) for individual in population]
print("Fitness Scores:", fitness_scores)
```

Step 4: Selection

- Combines fitness scores and population using zip(), pairs each board state with its fitness score.
- Sorts the population based on fitness (descending) because a higher score is better.
- Selects the top 50% of the population as parents for the next generation.
- These parents will be used to produce offspring in the next step (crossover).

```
# Select the best routes (parents) based on fitness
def select_parents(population, fitness_scores):
    # Select top 50% of the population
    sorted_population = [board for _, board in sorted(zip(fitness_scores,
population), reverse=True)]
    return sorted_population[:len(population)//2]

parents = select_parents(population, fitness_scores)
print("Selected Parents:", parents)
```

Step 5: Crossover

- Takes two parents (board) and produces an offspring.
- Randomly selects a subsequence from parent1 (from index start to end).
- Fills the rest of the child using parent2 in the order the queens appear, skipping any duplicates.

```
# Crossover function: single-point crossover with unique column
def crossover(parent1, parent2):
     point = random.randint(1, n - 2)  # Choose a crossover point
     child = parent1[:point] + parent2[point:]

    # Ensure unique column positions
    missing = set(range(n)) - set(child)
```

```
    duplicates = [col for col in child if child.count(col) > 1]
    for i in range(len(child)):
        if child.count(child[i]) > 1:
            child[i] = missing.pop()
    return child
```

- Selects random pairs of parents from the previously selected parents list.
- Generates population_size offspring through crossover.
- Fills the new population with children, replacing the previous population.

```
# Create new population using crossover
new_population = []
for i in range(population_size):
    parent1, parent2 = random.sample(parents, 2)
    child = crossover(parent1, parent2)
    new_population.append(child)
print("New Population after Crossover:", new_population)
```

Step 6: Mutation

- Randomly selects two unique indices (idx1 and idx2) from the board.
- Swaps the queens at these two rows.
- Returns the mutated board.
- Since it only swaps existing columns, it **preserves uniqueness**.

```
# Randomly swap two locations in a route
def mutate(route):
    idx1, idx2 = random.sample(range(n), 2)
    individual[idx1], individual[idx2] = individual[idx2], individual[idx1]
    return individual
```

- Loops through each route (child) in the new population.
- Applies mutation with a probability of 0.1 (10%): random.random() generates a random number between 0 and 1. If it is less than 0.1, mutation is applied.
- If mutation occurs, the board is slightly altered by swapping two queens.

```
mutation_rate = 0.1
for i in range(len(new_population)):
    if random.random() < mutation_rate:
        new_population[i] = mutate(new_population[i])
```

```
print("Population after Mutation:", new_population)
```

Step 7: Check Convergence Criteria

```
# Genetic Algorithm main function
def genetic_algorithm():
    population = [create_random_individual() for _ in
range(population_size)]
    generation = 0
    best_fitness = 0

    while best_fitness < 1.0 and generation < 100:
        fitness_scores = [calculate_fitness(ind) for ind in population]
        best_fitness = max(fitness_scores)
        print(f"Generation {generation} Best Fitness: {best_fitness}")

        # Check for optimal solution
        if best_fitness == 1.0:
            break

        # Selection
        parents = select_parents(population, fitness_scores)

        # Crossover
        new_population = [crossover(random.choice(parents),
random.choice(parents)) for _ in range(population_size)]

        # Mutation
        for i in range(len(new_population)):
            if random.random() < mutation_rate:
                new_population[i] = mutate(new_population[i])

        population = new_population
        generation += 1

    # Return the best solution
    best_individual = max(population, key=calculate_fitness)
    return best_individual, calculate_fitness(best_individual)
```

```
# Run the Genetic Algorithm
solution, fitness = genetic_algorithm()
print("Best Solution:", solution)
print("Best Fitness:", fitness)
```

Duty Scheduling Problem Using Genetic Algorithm

A hospital needs to create a weekly shift schedule for its staff, ensuring that there are enough staff members on duty during peak hours, while also giving each staff member proper rest.

Step 1: Define the Problem and Fitness Function

This fitness function **evaluates the quality of a given shift schedule** by calculating a **penalty score** based on two key factors:

1. **Shift Coverage:**
    - Checks if each shift has the required number of staff
    - If under-staffed, it adds a penalty of 10 per missing staff member for that shift.
    - The penalty increases as coverage becomes worse, making schedules with poor coverage less fit.
2. **Consecutive Shifts:**
    - Checks if any staff is assigned to back-to-back consecutive shifts (e.g., night shift followed by morning shift).
    - If a staff member works two consecutive shifts, it adds a penalty of 5 for each occurrence.
    - This discourages overworking employees and promotes work-rest balance.

```
import random
# Configuration
num_staff = 5  # Number of employees
num_shifts = 21  # 7 days * 3 shifts per day
max_shifts_per_staff = 7
required_staff_per_shift = 2
population_size = 10
mutation_rate = 0.1
max_generations = 1000
```

```
# Fitness function (lower is better)
def evaluate_fitness(schedule):
    penalty = 0

    # Check shift coverage
    for shift in range(num_shifts):
        assigned_count = sum(schedule[staff][shift] for staff in
range(num_staff))
        if assigned_count < required_staff_per_shift:
            penalty += (required_staff_per_shift - assigned_count) * 10  #
Understaffed penalty

    # Check consecutive shifts for each staff
    for staff in range(num_staff):
        for shift in range(num_shifts - 1):
            if schedule[staff][shift] == 1 and schedule[staff][shift + 1] ==
1:
                penalty += 5  # Penalty for consecutive shifts
    return penalty
```

Step 2: Initialize the Population

This function generates a random schedule (chromosome) as a 2D matrix → Rows: Staff, Columns: Shifts.

- Each cell is 1 if staff is assigned to a shift, otherwise 0.
- Randomly assigns 3 to max_shifts_per_staff shifts to each staff member.
- Returns a valid schedule representing one potential solution.

```
# Create a random schedule
def create_random_schedule():
    schedule = [[0] * num_shifts for _ in range(num_staff)]
    for staff in range(num_staff):
        assigned_shifts = random.sample(range(num_shifts), random.randint(3,
max_shifts_per_staff))
        for shift in assigned_shifts:
            schedule[staff][shift] = 1
    return schedule
```

Step 3: Selection

- Combines fitness scores and population using zip(), pairs each schedule with its fitness score.
- Sorts the population based on fitness (ascending) because a lower penalty is better.
- Selects the top 50% of the population as parents for the next generation.
- These parents will be used to produce offspring in the next step (crossover).

```
# Selection (Top 50%)
def select_parents(population, fitness_scores):
    sorted_population = [x for _, x in sorted(zip(fitness_scores, population))]
    return sorted_population[:len(population) // 2]
```

Step 4: Crossover.

- Single-point crossover → Combines two parent schedules to create a child.
- Random crossover point is selected across shifts (columns).
- Child inherits shifts before the point from parent1, and after the point from parent2.

```
# Crossover (Single point crossover)
def crossover(parent1, parent2):
    point = random.randint(0, num_shifts - 1)
    child = [parent1[i][:point] + parent2[i][point:] for i in range(num_staff)]
    return child
```

Step 5: Mutation

- Randomly selects one staff member.
- Swaps two randomly chosen shifts for that staff.
- Helps introduce small changes in the schedule.

```
# Mutation (Swap shifts for one staff)
def mutate(schedule):
    staff = random.randint(0, num_staff - 1)
    shift1, shift2 = random.sample(range(num_shifts), 2)
    schedule[staff][shift1], schedule[staff][shift2] = schedule[staff][shift2], schedule[staff][shift1]
```

```
    return schedule
```

Step 6: Convergence Criteria & GA Loop

```
# Initial population
population = [create_random_schedule() for _ in range(population_size)]

# Genetic Algorithm loop
for generation in range(max_generations):
    fitness_scores = [evaluate_fitness(schedule) for schedule in population]
    best_fitness = min(fitness_scores)
    print(f"Generation {generation + 1}, Best Fitness: {best_fitness}")
    parents = select_parents(population, fitness_scores)
    new_population = []
    while len(new_population) < population_size:
        parent1, parent2 = random.sample(parents, 2)
        child = crossover(parent1, parent2)
        if random.random() < mutation_rate:
            child = mutate(child)
        new_population.append(child)

    population = new_population
# Best schedule
best_schedule = population[fitness_scores.index(min(fitness_scores))]
print("\nBest Schedule (Staff x Shifts):")
for staff in range(num_staff):
    print(f"Staff {staff + 1}: {best_schedule[staff]}")
```

LAB TASKS

TASK: 01

Use beam search to predict the best move in a game of chess by evaluating a subset of the most promising moves. Limit the search depth and beam width for efficiency.

Input: Current board state, beam width, and depth limit.
Output: Best move sequence and its evaluation score.

TASK: 02

Design a hill climbing algorithm to find the shortest delivery route among a set of locations. Make incremental changes to the route and accept them only if they reduce the total distance.

Input: List of coordinates for delivery points.
Output: Optimized route and the total distance covered.

TASK: 03

Implement a genetic algorithm to solve the TSP (Traveling Salesman Problem) for a given set of 10 cities.